

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments. (BL2, BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5
Marks:25

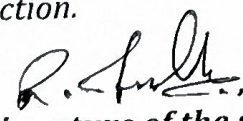
Total

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 - Exemplary (Full Marks)	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Marks Evaluation:

Question No.	Conceptual Understanding (30%)	Accuracy of Answers (25%)	Application of Knowledge (20%)	Completeness of Response (15%)	Clarity & Presentation (10%)	Total Marks
Q1						
Q2						
Q3						
Q4						
Q5						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name: R. Indhu

Reg no: 192524832

Course: Operating Systems

1. Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.

(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.

(b) Draw the structure of an Operating system suitable for the showing how resources are managed.

(c) Write a sequence of system calls required to create and execute a new process in such an environment.

2. Consider an Operating System managing multiple files & devices for concurrent users.

(a) Categorize system calls used for the device management.

(b) Illustrate how the OS handles file operations diagram.

(c) Write a pseudo-code or C-based system call sequence to open.

3. Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the types of OS and explain why it is suitable.

(b) Draw a diagram representing the interaction b/w hardware, kernel.

(c) List & explain system calls required for process synchronization.

4. A distributed computing system consists of multiple interconnected computers sharing.

- Classify the type of OS used & compare it with OS.
- Draw the architecture of distributed OS showing.
- Write the sequence of system calls involved in inter-process communications.

5. Consider a scenario where an OS must manage process scheduling memory allocation.

- Explain the major function of an OS handling these tasks.
- Draw a block diagram representing OS components and their interactions.
- Write system call examples for memory allocation & device management.

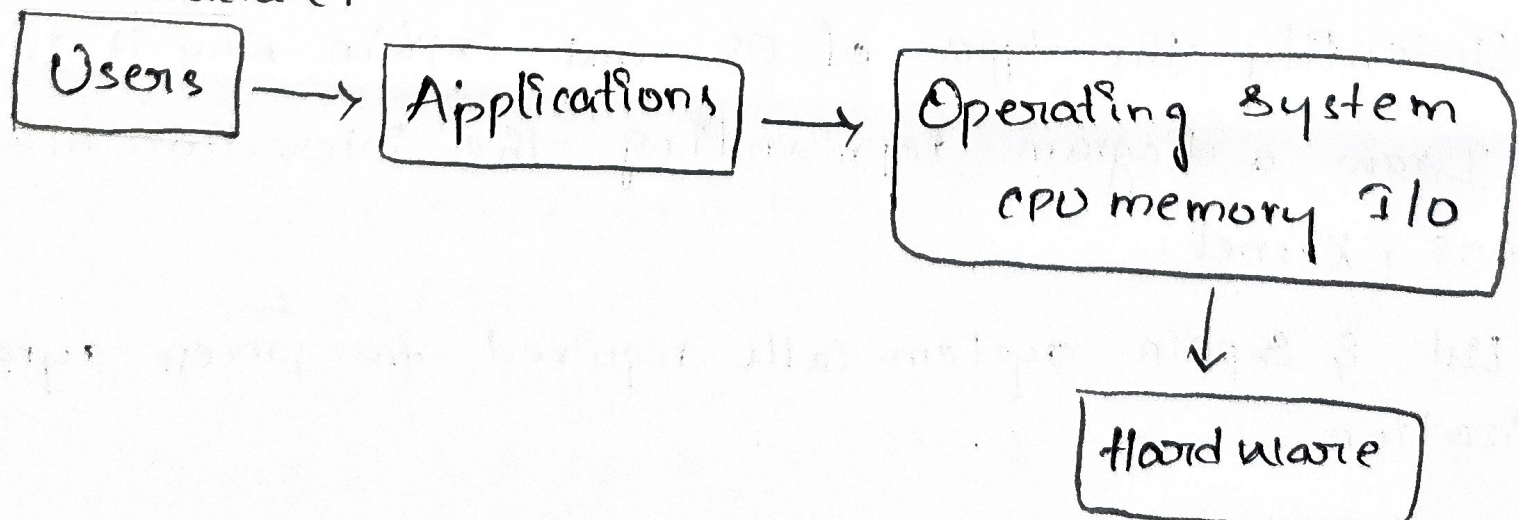
1.

(a) Types of Operating System:

- * Multi user OS - Allows multiple users to work at the same time.
- * Multitasking OS - Runs multiple processes simultaneously.
- * Time-sharing OS - shares CPU time among users and process.

(b)

OS structure:



(c) System Call sequence:

* fork()

* exec()

* wait()

* exit()

2.

(a) System Calls for file and Device Management

File Management:

open(),

read(),

write(),

close(),

lseek()

Device Management:

ioctl(),

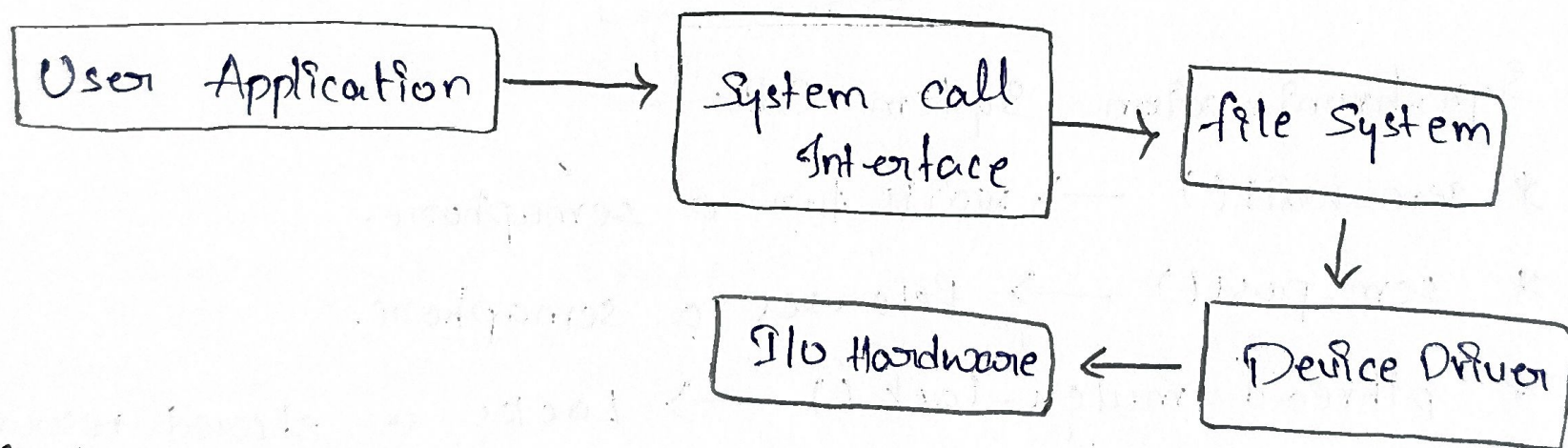
read(),

write(),

open(),

close()

(b) Layered file Operating Structure



(c) C-based System Call Sequence

```
fd = open("data.txt", O_RDONLY);
```

```
read(fd, buffer, 100);
```

```
close(fd);
```

* open() → opens the file

* read() → Reads the file

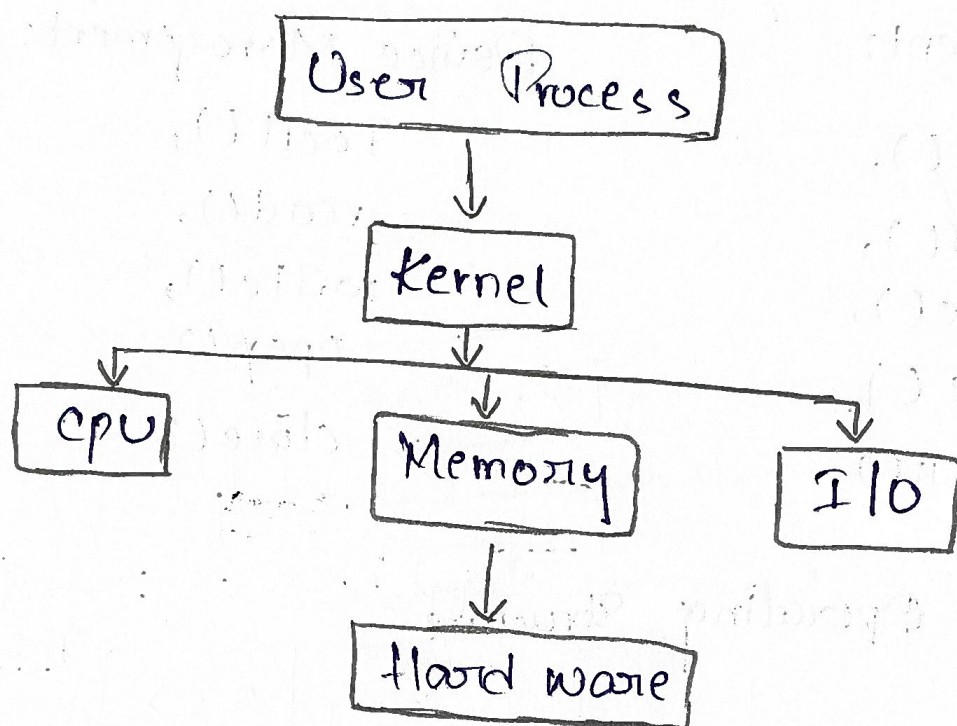
* close() → closes the file

3.

(a) Type of OS:

Real-Time Operating System (RTOS) is suitable for industrial machinery because it suitable provides fast and predictable responses within strict time limits.

(b) Interaction Diagram:



(c) Synchronization System Calls:

- * `sem_wait()` → waits for a semaphore.
- * `sem_post()` → Releases a semaphore.
- * `pthread_mutex_lock()` → Locks a shared resource.
- * `pthread_mutex_unlock()` → Unlocks the resource.
- * `pthread_cond_wait()` → waits for a condition.

4.

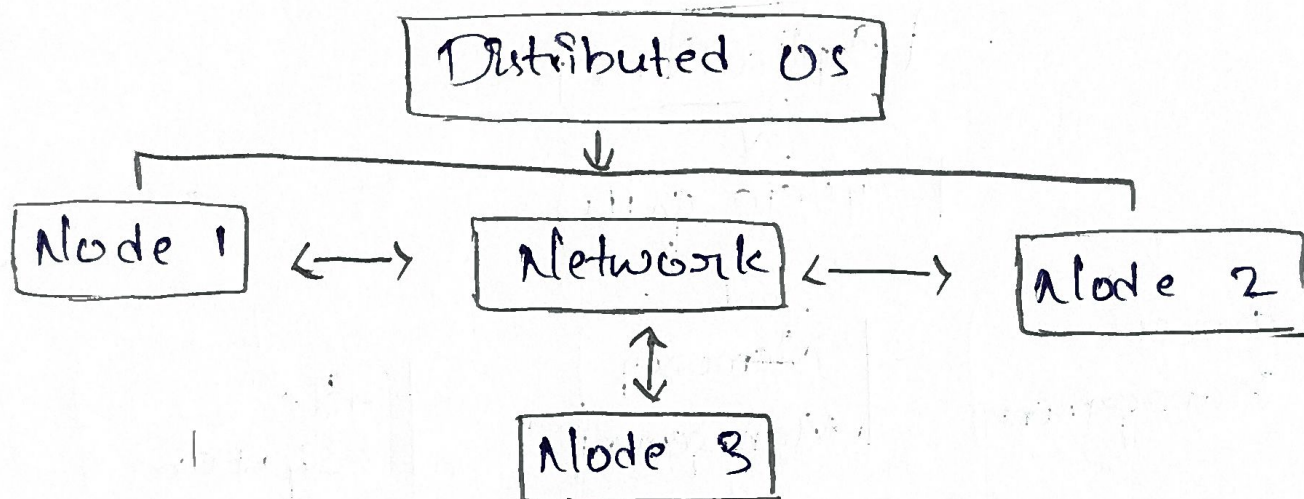
(a) Distributed OS:

A Distributed Operating System manages multiple interconnected computers as one integrated system.

Comparison:

- Distributed OS: Resources are shared across multiple networked computers.
- Centralized OS: Resources are managed mainly by one central computer.

(b) Distributed OS Architecture



(c) IPC System Call Sequence:

`pipe()` → `write()` → `read()` → `close()`

* `pipe()` → creates communication channel

* `write()` → sends data.

* `read()` → Receives data.

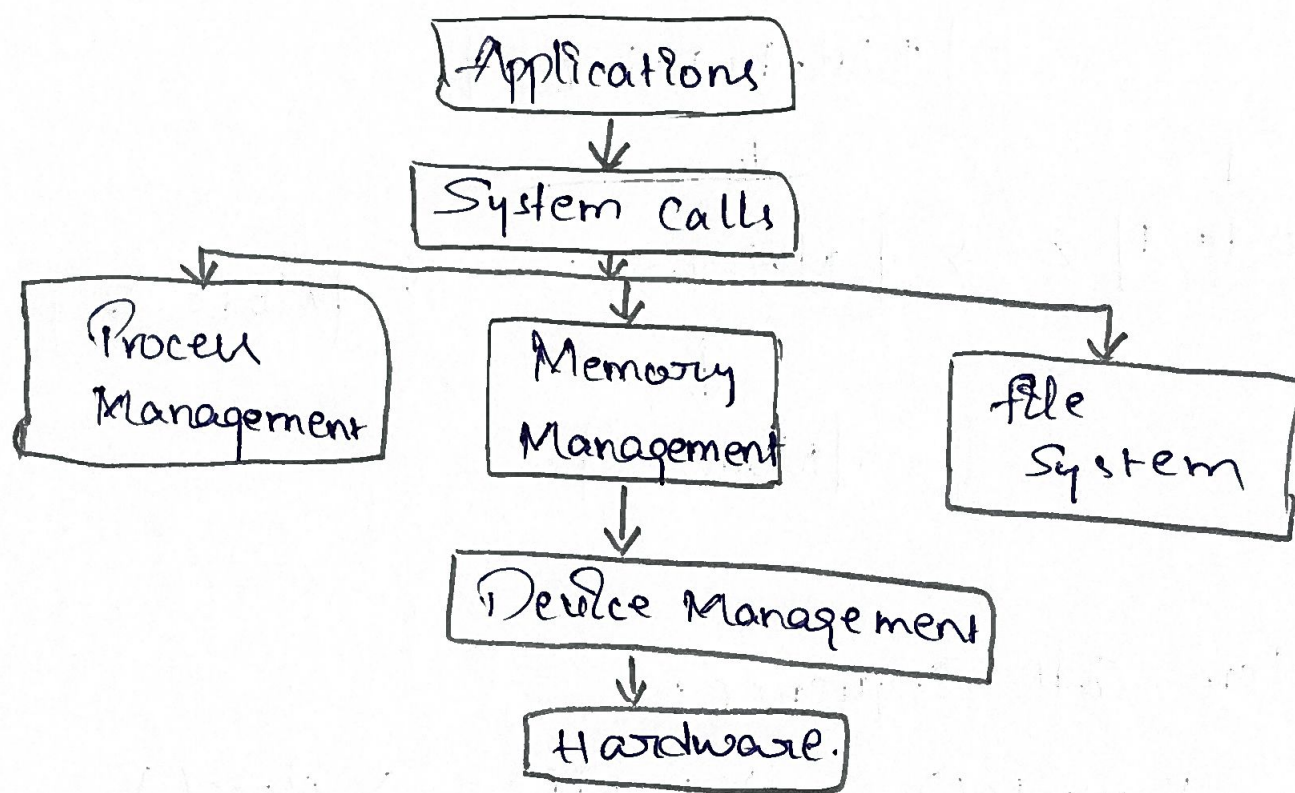
* `close()` → closes communication channel.

5.

(a) Major functions of OS

- * Process Scheduling: Allocates CPU time to processes.
- * Memory Management: Allocates and frees memory.
- * Device Management: Controls input/output memory.
- * File management: files are organized.
- * Security: protects system resources.

(b) OS Components:



(c) System Call Examples:

Memory Management

`malloc();`

`free();`

`mmap();`

Device Management:

`open();`

`read();`

`write();`

`ioctl();`

`close();`